

INFORME DE LABORATORIO — TALLER 1

Construcción de Pantalla Básica con StatefulWidget, setState() y Control de Versiones con Git Flow

Estudiante:	Michael Stiven Vasco Cárdenas
Código Estudiantil:	230231047
Asignatura:	Electiva Profesional I
Docente:	Ingeniería de Sistemas
Repositorio GitHub:	https://github.com/BreiK-at/Taller-flutter
Rama del Taller:	feature/taller1
Fecha:	Septiembre 2026

Tuluá, Valle del Cauca, Colombia
2026

1. Objetivo del Taller

El objetivo primordial de esta práctica es diseñar y construir una pantalla funcional en **Flutter** implementando la gestión del estado reactivo mutable con un **StatefulWidget** y la invocación de **setState()**. Adicionalmente, se consolida la aplicación rigurosa de buenas prácticas de control de versiones con **Git Flow**, empleando ramas base (main, dev), ramas de taller (feature/taller1) y flujos de integración mediante Pull Requests.

2. Fundamentación Técnica: StatefulWidget y setState()

En Flutter, la interfaz es declarativa y reactiva. Los widgets representan la configuración inmutable de la vista. Para gestionar componentes dinámicos con ciclo de vida mutable, se utiliza **StatefulWidget**.

2.1 Arquitectura de StatefulWidget:

- **Widget (Clase inmutable):** Hereda de **StatefulWidget** y solo actúa como plantilla externa. Sobreescribe **createState()** para instanciar la clase de estado asociada.
- **State (Clase de lógica y estado mutable):** Hereda de **State<T>**, almacena los datos en memoria a lo largo del tiempo y describe el árbol visual dentro del método **build(BuildContext context)**.

2.2 Funcionamiento interno de setState():

El método **setState(VoidCallback fn)** es el núcleo de la reactividad local en Flutter. Su mecanismo opera en 4 fases:

1. **Mutación de Estado:** Ejecuta el callback sincrónico pasado por parámetro, actualizando las variables (como **_titulo** y **_contadorClicks**).
2. **Marcado de Elemento:** Notifica al framework que este nodo del árbol de elementos está **dirty** (modificado).
3. **Programación de Frame:** El motor de renderizado de Flutter agenda la ejecución del método **build()** en el siguiente refresco de pantalla.
4. **Reconciliación (Diffing):** Flutter compara el nuevo árbol con el anterior y redibuja en la GPU exclusivamente los widgets que sufrieron alteraciones, preservando un rendimiento óptimo de 60/120 FPS.

2.3 Fragmento de Código Implementado:

```
void _alternarTitulo() {
  setState(() {
    _tituloModificado = !_tituloModificado;
    _titulo = _tituloModificado ? '¡Título cambiado!' : 'Hola, Flutter!';
    _contadorClicks++;
  });

  ScaffoldMessenger.of(context).hideCurrentSnackBar();
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: const Row(children: [
        Icon(Icons.check_circle, color: Colors.white),
        SizedBox(width: 12),
        Text('Título actualizado', style: TextStyle(fontWeight: FontWeight.bold)),
      ]),
      backgroundColor: Colors.indigo.shade700,
      behavior: SnackBarBehavior.floating,
      duration: const Duration(seconds: 2),
    ),
  );
}
```

3. Evidencias de la Aplicación en Ejecución

A continuación se presentan las capturas de la aplicación en ejecución sobre el escritorio nativo Linux, evidenciando el cumplimiento de los requerimientos visuales y funcionales:

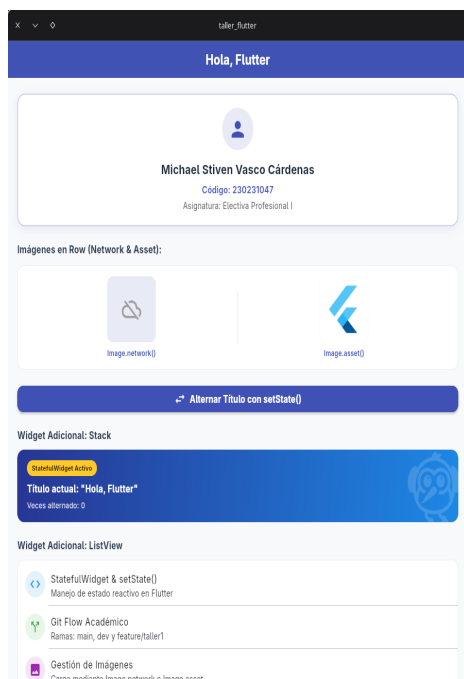


Figura 1: Estado Inicial

AppBar con 'Hola, Flutter', tarjeta centrada con datos del estudiante y Row de imágenes (Image.network + Image.asset).

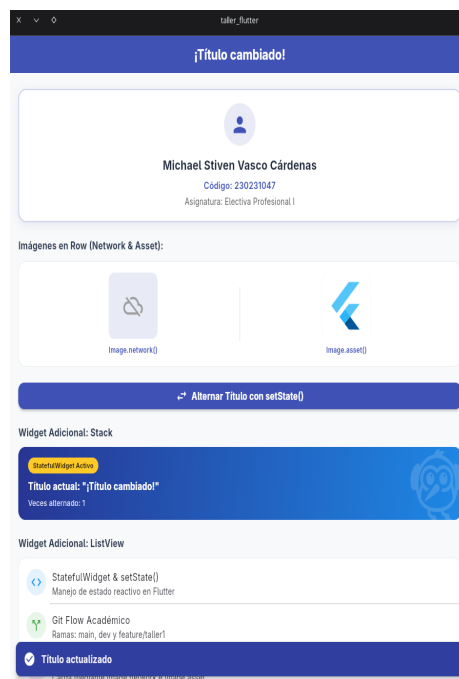


Figura 2: Estado Tras Botón + Snackbar

Título reactivo '¡Título cambiado!' actualizado con setState() y Snackbar flotante con 'Título actualizado'.

Detalle de Componentes Evaluados:

- **AppBar Dinámica:** Título sincronizado con la variable reactiva `_titulo`.
- **Identificación Centrada:** Tarjeta decorada con nombre **Michael Stiven Vasco Cárdenas**, código **230231047** y materia **Electiva Profesional I**.
- **Row con Imágenes:** Integración de `Image.network()` (remota) e `Image.asset('assets/images/flutter_logo.png')` (local).
- **Botón de Acción:** `ElevatedButton` con icono que invoca `_alternarTitulo()` con retroalimentación visual.

4. Widgets Adicionales Implementados

Se implementaron **tres widgets adicionales** (superando el mínimo de 2 requeridos):

- **Container:** Tarjeta decorada con bordes redondeados (`BorderRadius.circular(16)`), sombra difusa y padding interno.
- **Stack:** Superposición de badge informativo ('*StatefulWidget Activo*'), texto y marca de agua sobre gradiente.
- **ListView:** Lista vertical con 4 elementos (`ListTile`), cada uno con avatar temático, título y subtítulo.

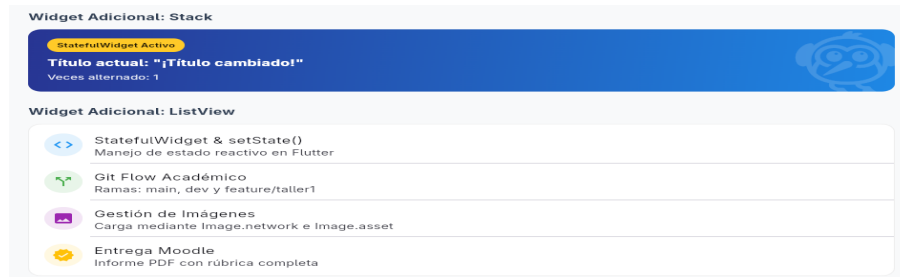


Figura 3: Widgets Adicionales: Stack con gradiente interactivo y ListView informativo estructurado.

5. Evidencia del Flujo de Trabajo en Git (Git Flow)

Flujo de ramas y Pull Requests implementado de acuerdo al diagrama oficial del docente:

Rama	Propósito / Rol	Estado en Repositorio
main	Rama estable de producción.	Integrada vía PR #2.
dev	Rama base de desarrollo continuo.	Integrada vía PR #1.
feature/taller1	Rama de desarrollo del Taller 1.	Commits y PR originados aquí.

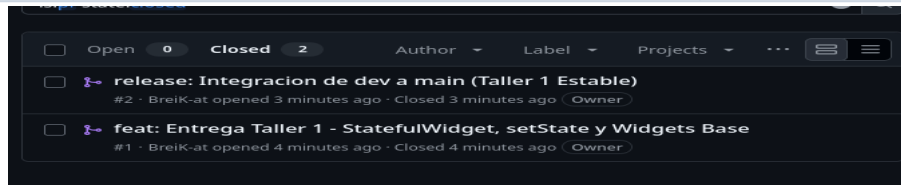


Figura 4: Pull Requests en GitHub: PR #1 (feature/taller1 → dev) y PR #2 (dev → main) cerrados y fusionados (Merged).

6. Enlace de Repositorio y Conclusiones

- **URL Repositorio Público:** <https://github.com/BreiK-at/Taller-flutter>
- **Rama del Taller:** feature/taller1
- **Conclusión:** Se validó exitosamente la reactividad en Flutter con `StatefulWidget` y `setState()`, comprobando que la arquitectura desacoplada de Widgets y State garantiza un renderizado eficiente. Asimismo, se demostró la efectividad de Git Flow para la entrega controlada de código.